

Project Presentation of CO-OP Project at Industry (Module-2) (22CS421)

On

**A Reliability Control Framework for Robust Multi-Agent LLM Systems:
Managing Workflows in Large Language Model Systems**

Jasneet Arora

2210990451

Supervised By

Dr. Gurpreet Singh

Department of Computer Science and Engineering,
Chitkara University, Punjab

- Introduction
- Objectives
- Framework Architecture
- Tools and Technologies
- Implementation: Recovery Algorithms
- Implementation: Safety & Monitoring
- Results
- Conclusion & Future Scope

- **Domain:** Multi-agent LLM systems increasingly deployed in enterprise — fraud detection, analytics, customer service
- **Problem:** Real-world failures frequent — coordination breakdowns, tool misuse, output inconsistencies, workflow interruptions
- **Existing Frameworks:** AutoGen, MetaGPT, LangChain, CAMEL treat reliability as an afterthought
- **Gap:** No built-in fault detection, automated recovery, or integrated safety validation in current systems
- **Need:** Enterprise robustness against degraded agents, network instability, context overflows — without human supervision

- **Architecture:** Design a seven-layer framework (ARCF) with reliability supervision as a first-class concern
- **Monitoring:** Build a real-time engine using Multi-Agent System Failure Taxonomy (MAST)
- **Recovery:** Implement four algorithms — adaptive retry, hierarchical fallback, checkpoint recovery, circuit breaker isolation
- **Observability:** Integrate OpenTelemetry GenAI semantic conventions
- **Safety:** Apply Constitutional AI principles for three-tier output verification
- **Evaluation:** Validate through simulation with controlled fault injection at 5%, 10%, 20% rates

Layer	Name	Function
L1	Foundation Model Layer	LLM/retrieval encapsulation, telemetry capture
L2	Data Operations Layer	State management, vector DBs, event ledgers
L3	Agent Framework Layer	Reasoning-action agents with health scoring
L4	Orchestration Layer	DAG-based workflow, dependency resolution
L5	Reliability Control Layer	Failure detection, recovery, reliability scoring
L6	Observability & Security	Traces, safety monitoring, injection detection
L7	Integration & API Layer	Enterprise APIs, monitoring dashboard

Core Contribution:

L5 (Reliability Control Layer) acts as supervisory plane over the entire stack

- **Orchestration Frameworks:** AutoGen, MetaGPT, LangChain, CAMEL — ARCF operates above these without modifying core logic
- **Observability Standard:** OpenTelemetry GenAI Semantic Conventions — standardized tracing for LLM calls, token usage, agent latency; compatible with Datadog
- **Safety Framework:** Constitutional AI (Anthropic) — principle-based output evaluation for alignment and safety
- **Workflow Representation:** Directed Acyclic Graphs (DAGs) — dependency resolution, parallel execution, failure localization
- **Development Stack:** Python + FastAPI — asynchronous service interfaces and prototype implementation
- **Telemetry Collection:** OpenTelemetry Python SDK — real-time metrics and trace processing

- **Adaptive Exponential Backoff**
 - **Mechanism:** Retry intervals scale exponentially per attempt
 - **Adaptation:** Backoff factor adjusts — 2.0 (success rate <50%), 1.5 (<75%), 1.0 (otherwise)
- **Hierarchical Fallback Agent Selection**
 - **Scoring:** Reliability history (0.40) + capability match (0.30) + seniority (0.20) + cost (0.10)
 - **Context Transfer:** Execution state handed to fallback agent — no redundant recomputation
- **Checkpoint-Based State Recovery**
 - **Snapshots:** Workflow state saved at regular intervals during execution
 - **Rollback Condition:** Triggered only if re-execution cost < remaining workflow value
- **Circuit Breaker Isolation**
 - **States:** Closed → Open (failure threshold breached) → Half-Open (probe tasks) → Closed (stable)
 - **Purpose:** Prevents persistent agent failures from cascading across the workflow

- **Three-Tier Safety Pipeline:**
 - **Tier 1** — Schema Validation: Confirms output matches expected structure
 - **Tier 2** — Constitutional AI: Rule-based filters + principle-based safety assessment
 - **Tier 3** — Logical Consistency: Coherence with task context and prior agent outputs
- **MAST-Aware Monitoring Engine:**
 - **Specification Failures:** Timeouts, context overflows, schema violations, tool misuse
 - **Inter-Agent Failures:** Communication errors, output conflicts, role violations, deadlocks
 - **Verification Failures:** Premature termination, infinite loops, output quality degradation

Results

Metric	Baseline	ARCF
Workflow Completion Rate	78%	96%
Failure Recovery Rate	—	89%
MAST Coverage Rate	—	93%
Retry Success Rate	—	84%
Fallback Success Rate	—	79%
Checkpoint Recovery Rate	—	91%
Safety Block Rate	—	1.2%
Response Latency (P95)	1.8s	2.1s
Observability Overhead	—	3–4%

- **Reliability Gain:** +18 percentage points in workflow completion rate
- **Autonomy:** 89% of failures recovered without human intervention
- **Overhead:** Only 300ms latency increase — acceptable for enterprise deployment

- **Conclusion:**

- **Reliability:** ARCF raises workflow completion from 78% → 96%
- **Recovery:** Autonomously handles 89% of failures; covers 93% of known MAST failure modes
- **Overhead:** Adds only 3–4% latency — production-suitable
- **Validation:** Confirms architectural reliability supervision is necessary and effective for multi-agent LLM systems

- **Future Scope:**

- **Deployment:** Live enterprise testing under real operational complexity
- **Detection:** Learned failure classifiers to replace rule-based MAST engine
- **Verification:** Formal methods for recovery state transition correctness
- **Concurrency:** Support for parallel and asynchronous agent workflows
- **Adaptivity:** Self-improving agents that update behaviour from reliability history

Thank You